



PROYECTO INTERMODULAR

CourtHub

Sistema de Gestión Integral de Reservas para Centros Deportivos

Grado:

Desarrollo de Aplicaciones Multiplataforma

Institución:

IES Comercio

Autor:

Alejandro Daniel Sahonero Ampuero

Tutor:

Óscar Lázaro Moreno

Mayo de 2026

ÍNDICE

ÍNDICE DE ILUSTRACIONES.....	iii
ÍNDICE DE TABLAS.....	iii
1. Introducción.....	1
1.1. Contexto del sector.....	1
1.2. Identificación del problema.....	1
1.3. Justificación del nombre.....	2
2. Objetivos.....	3
2.1 Objetivo general.....	3
2.2 Objetivos específicos.....	3
3. Metodologías y Fases del proyecto.....	4
3.1. Metodología de trabajo.....	4
3.2. Control de versiones y flujo de trabajo.....	4
3.3. Cronograma y desarrollo de fases.....	5
3.3.1 Fase 1: Infraestructura base y desafíos de compatibilidad (Semanas 1-2).....	5
3.3.2 Fase 2: Diseño UI/UX y Prototipado (Semanas 2-3).....	6
3.3.3 Fase 3: Desarrollo de la lógica de datos (Semanas 3-4).....	7
3.3.4 Fase 4: Funcionalidades Core y Automatización (Semanas 4-5).....	8
3.3.5 Fase 5: Depuración, pulido y documentación (Semana 6).....	8
4. Diseño del sistema.....	9
4.1. Arquitectura de software.....	9
4.1.1. Capa de dominio.....	9
4.1.2. Capa de datos.....	10
4.1.3. Capa de Presentación.....	11
4.2 Stack tecnológico.....	12
4.3. Modelo de datos y persistencia.....	13
5. Implementación y configuración.....	16
5.1. Seguridad y Autorización del servidor.....	17
5.1.1. Protección de integridad de perfiles y roles.....	17
5.2. Desarrollo del Sistema de Acceso.....	18
5.3. Resolución de problemas y lógica de negocio crítica.....	19
6. Pruebas y control de calidad (QA).....	20
6.1. Pruebas Unitarias.....	21
6.2 Entorno de Pruebas.....	23
6.3 Verificación de la Persistencia y Acceso Offline.....	23

6.4 Pruebas de Integración y Validación de Acceso.....	23
7. Ampliaciones y posibles mejoras	24
7.1 Integración real de pasarela de pagos	24
7.2 Escalabilidad y modelo Multi-tenant	24
7.3 Internacionalización y accesibilidad técnica	24
7.4 Automatización total del ciclo de vida (CI/CD)	25
8. Conclusión.....	25
8.1 Análisis de resultados frente a objetivos iniciales	25
8.2 Valoración personal y aprendizaje técnico	26
9. Bibliografía	27

ÍNDICE DE ILUSTRACIONES

Ilustración 1: Gráfica del control de versiones (Git). Fuente: Elaboración propia	5
Ilustración 2: Comparativa Figma (Izquierda) vs Diseño final (Derecha). Fuente: Elaboración propia	7
Ilustración 3: Ejemplo de un “Use Case” escrito en el lenguaje de programación kotlin: Fuente: Elaboración propia.....	10
Ilustración 4: Arquitectura MVVM. Fuente: medium.com	11
Ilustración 5: Diagrama de datos de Firebase	14
Ilustración 6: Diagrama de datos local.....	15
Ilustración 7: Security Rules de Firestore	17
Ilustración 8: Flujo del sistema de roles	18
Ilustración 9: Diagrama de flujo del proceso de validación de acceso	19
Ilustración 10: Ejemplo del repositorio. Fuente: Elaboración propia.....	20
Ilustración 11: Tests realizados en Android Studio.....	22
Ilustración 12: Captura del panel de resultados de los tests de la ilustración anterior. Fuentes: Elaboración propia	22

ÍNDICE DE TABLAS

Tabla 1: Casos de usos implementados	11
Tabla 2: Comparación de responsabilidades en la persistencia	16

1. Introducción

Hacer una aplicación no es solo escribir código, sino buscar una forma real de ayudar a que los negocios funcionen mejor. Hoy en día, tener un sistema digital no es un lujo, es algo básico para que un centro deportivo esté bien organizado. Este proyecto se centra en crear una herramienta moderna y fácil de usar que solucione los problemas típicos que ocurren cuando se gestionan pistas y reservas a mano. El objetivo es que CourtHub sea una plataforma sólida que conecte bien a los administradores con los clientes, usando la tecnología actual para que todo el proceso sea más rápido y seguro.

1.1. Contexto del sector

Ahora mismo, casi cualquier servicio que usamos funciona a través de una aplicación. Sin embargo, en Logroño y en el resto de La Rioja, todavía hay una diferencia muy grande entre los centros deportivos. Mientras que los gimnasios o clubes muy grandes tienen sus propios programas caros, los centros más pequeños siguen usando métodos antiguos, como apuntar cosas en papel o usar herramientas que no están pensadas para eso. CourtHub nace en 2026 para cerrar esa brecha, ofreciendo una aplicación nativa de Android que dé a estos centros una imagen profesional y una gestión moderna sin complicaciones.

1.2. Identificación del problema

Al analizar cómo funcionan los centros deportivos que aún dependen de una gestión manual, el problema no es solo la lentitud, sino el coste que supone tener el centro abierto. Actualmente, estos centros se encuentran con tres obstáculos principales que limitan su crecimiento:

- **Costes operativos y horarios limitados:** Para que un usuario pueda reservar y entrar a una pista, el centro necesita tener a una persona físicamente en la recepción para atenderle y abrirle la puerta. Esto hace que sea imposible ofrecer un servicio de 24 horas, ya que, si el centro quiere abrir de madrugada o domingos por la tarde, el coste del personal suele superar el beneficio de las reservas. CourtHub soluciona esto permitiendo que el centro funcione solo, mientras que el personal físico puede centrarse en tareas de mayor valor o en ayudar a colectivos que no dominan la tecnología, como las personas de mayor edad que siempre necesitarán que alguien les indique físicamente a qué pista ir o cómo reservar. Al automatizar el acceso de los clientes "tecnológicos", el personal del centro no tiene que estar perdiendo el tiempo abriendo y cerrando puertas, pudiendo dar una atención más personalizada a quien realmente la necesite.
- **Validación de acceso manual:** Al no haber un sistema automático, la comprobación de quién tiene permiso para entrar depende totalmente del recepcionista. Si hay mucha gente o el encargado está haciendo otra tarea, el acceso se vuelve un caos y es fácil que se cuelen personas sin reserva o que se produzcan errores humanos al anotar las horas.

- **Dependencia de la atención presencial:** El mayor fallo es que el negocio solo genera ingresos si hay alguien allí. Si el centro quiere que un cliente pueda ir a jugar a las 7 de la mañana antes de trabajar, alguien tiene que madrugar para abrirle. Esto quita comodidad al usuario y eficiencia al dueño del centro.
- **Dependencia del internet:** La estructura física de los polideportivos suele bloquear las señales de datos, convirtiendo la conectividad en un factor poco fiable para un sistema de acceso que debe ser instantáneo.

CourtHub soluciona esto permitiendo una atención 24/7 sin necesidad de personal en la entrada. La idea es que el centro instale un lector de códigos y el usuario, tras reservar desde su casa, llegue y entre directamente escaneando su QR¹. Para poder demostrar esto en la exposición del TFG sin necesidad de comprar un lector profesional, he programado una función de escaneo dentro del perfil de "Administrador" de la aplicación. De esta forma, simulo que el móvil del administrador es el escáner del centro para que el tribunal vea en tiempo real cómo el código QR o el código OTP² validan la entrada correctamente. Adicionalmente, se ha implementado una estrategia de persistencia híbrida. El uso de bases de datos locales resulta crítico para mitigar las deficiencias de cobertura habituales en los centros deportivos, donde la propia estructura física y los muros de las instalaciones suelen interferir con la señal inalámbrica, garantizando así un acceso fluido y fiable incluso en condiciones de red inestables

1.3. Justificación del nombre

Antes de entrar en los detalles técnicos, es importante explicar la identidad del proyecto. El nombre CourtHub no es casualidad; nace de la unión de dos conceptos en inglés que definen la filosofía de la aplicación:

- **Court (Pista):** Hace referencia directa al objeto central del negocio: la pista deportiva (ya sea de pádel, tenis, fútbol, etc.).
- **Hub (Centro/Espacio común):** Se utiliza en tecnología para definir un punto de conexión o un espacio central donde se gestionan las pistas.

Al juntarlos, CourtHub se presenta como una "comunidad de pistas" o un espacio común donde el usuario tiene a su disposición una recopilación de instalaciones. El nombre transmite la idea de que el cliente puede encontrar la pista que quiera y tenerla disponible cuando quiera, eliminando las barreras de horarios y esperas.

¹ (Quick Response) Código de barras bidimensional capaz de almacenar datos que pueden ser leídos a gran velocidad por dispositivos móviles para realizar acciones automáticas.

² (One-Time Password) Sistema de seguridad que genera una contraseña válida únicamente para una sesión o transacción, quedando invalidada automáticamente tras su uso o al expirar un tiempo breve.

2. Objetivos

Tras analizar los problemas de los centros deportivos que dependen de la atención presencial, CourtHub busca la optimización del modelo operativo de los centros deportivos mediante la automatización de procesos críticos, es por eso que se marca una serie de metas para transformar ese modelo de negocio en uno más eficiente y moderno.

2.1 Objetivo general

El objetivo principal de este proyecto es desarrollar una aplicación móvil integral Android nativo que permita la gestión automatizada y autónoma de centros deportivos. Se busca que el centro pueda funcionar bajo un modelo "24/7" sin necesidad de personal físico en la entrada, facilitando el acceso a los usuarios mediante tecnología de validación digital y permitiendo al administrador gestionar todo el negocio desde su propio dispositivo móvil.

2.2 Objetivos específicos

Para alcanzar el objetivo general, se han definido las siguientes metas técnicas y de negocio:

1. **Automatización de accesos:** Implementar el uso de códigos QR (Quick Response) y sistemas OTP (One-Time Password) para permitir la entrada autónoma del usuario.
2. **Gestión por roles en una sola plataforma:** Crear una única aplicación que se adapte según quién inicie sesión. Si es un cliente, podrá reservar; si es un administrador, podrá gestionar pistas y usuarios desde cualquier lugar, sin tener que estar sentado en una oficina frente a un ordenador.
3. **Seguridad y control de tiempos:** Desarrollar una lógica de códigos dinámicos que caduquen automáticamente. La idea principal es evitar que un código solo sirva para la hora reservada y no pueda ser reutilizado ni compartido de forma fraudulenta.
4. **Optimización de recursos (24/7):** Demostrar que es posible mantener un centro deportivo abierto a cualquier hora del día reduciendo al mínimo los costes operativos, permitiendo que el administrador tenga el control total de lo que ocurre en su centro en tiempo real desde su bolsillo.
5. **Simulación de hardware profesional:** Incorporar un módulo de escaneo integrado en el rol de administrador que permita emular el comportamiento de un lector de acceso profesional, facilitando la demostración de la integridad del sistema durante la fase de exposición del proyecto.

3. Metodologías y Fases del proyecto

Para asegurar que CourtHub fuera un proyecto viable y bien estructurado, no bastaba con escribir código de forma aislada. Ha sido necesario aplicar una metodología de trabajo que permitiera organizar las ideas, gestionar los cambios y, sobre todo, garantizar que cada nueva funcionalidad no rompiera lo que ya estaba funcionando. A continuación, se detalla el marco metodológico y el conjunto de herramientas que han permitido desarrollar una aplicación funcional que resolviera el problema planteado.

3.1. Metodología de trabajo

Se ha optado por una metodología Scrum simplificada, dividiendo el desarrollo en ciclos cortos o "sprints". Esta forma de trabajar ha permitido que la aplicación fuera creciendo de manera modular: primero se consolidó la base del sistema y el inicio de sesión, para luego ir añadiendo las funciones de reserva y, finalmente, el sistema de acceso mediante QR. Trabajar así ha facilitado la detección temprana de errores y ha permitido adaptar el diseño a medida que surgían nuevas necesidades funcionales.

3.2. Control de versiones y flujo de trabajo

El uso de Git (sistema de control de versiones) y GitHub (la plataforma web) es lo que realmente eleva la calidad técnica de CourtHub. No se han usado solo para almacenar el código, sino para gestionar un proyecto Open Source (código abierto). En lugar de simplemente subir archivos, se ha implementado un flujo de trabajo profesional para proteger la integridad del código:

- **Filosofía Open Source:** Al publicar el proyecto en GitHub, permite que desarrolladores de cualquier parte del mundo puedan revisar el código, sugerir mejoras o incluso colaborar en el desarrollo. Es la forma en la que se construye el software más importante del mundo hoy en día.
- **El flujo de las Pull Requests (PR):** Esta es la herramienta más potente de GitHub. Se llama "Pull Request" porque un desarrollador, tras hacer una mejora, le pide al dueño del proyecto que "tire" (pull) de sus cambios para incorporarlos al código principal.
 - **Para qué sirven:** Una PR abre un espacio de debate donde se pueden ver exactamente qué líneas han cambiado, comentar fallos y proponer correcciones antes de aceptar el cambio. En CourtHub, esto ha servido para asegurar que ninguna funcionalidad nueva dañara la rama principal (*main*), garantizando que la aplicación siempre fuera estable.
- **Estructura de ramas:** Se ha utilizado una rama secundaria denominada *develop* para el trabajo diario, manteniendo la rama *main* exclusivamente para versiones estables

y libres de errores. Esto garantiza que cualquier fallo durante las pruebas no afecte a la versión principal de la aplicación.

- **Documentación (README):** El repositorio cuenta con un archivo *README* detallado que explica la finalidad del proyecto y las tecnologías utilizadas. Esto es vital para que cualquier otro desarrollador pueda entender y colaborar en el proyecto.
- **Gestión de secretos:** Siguiendo las mejores prácticas de seguridad, las claves sensibles (como el Web Client ID de Google) no se han subido al repositorio, sino que se han gestionado mediante archivos locales inyectados en tiempo de compilación.

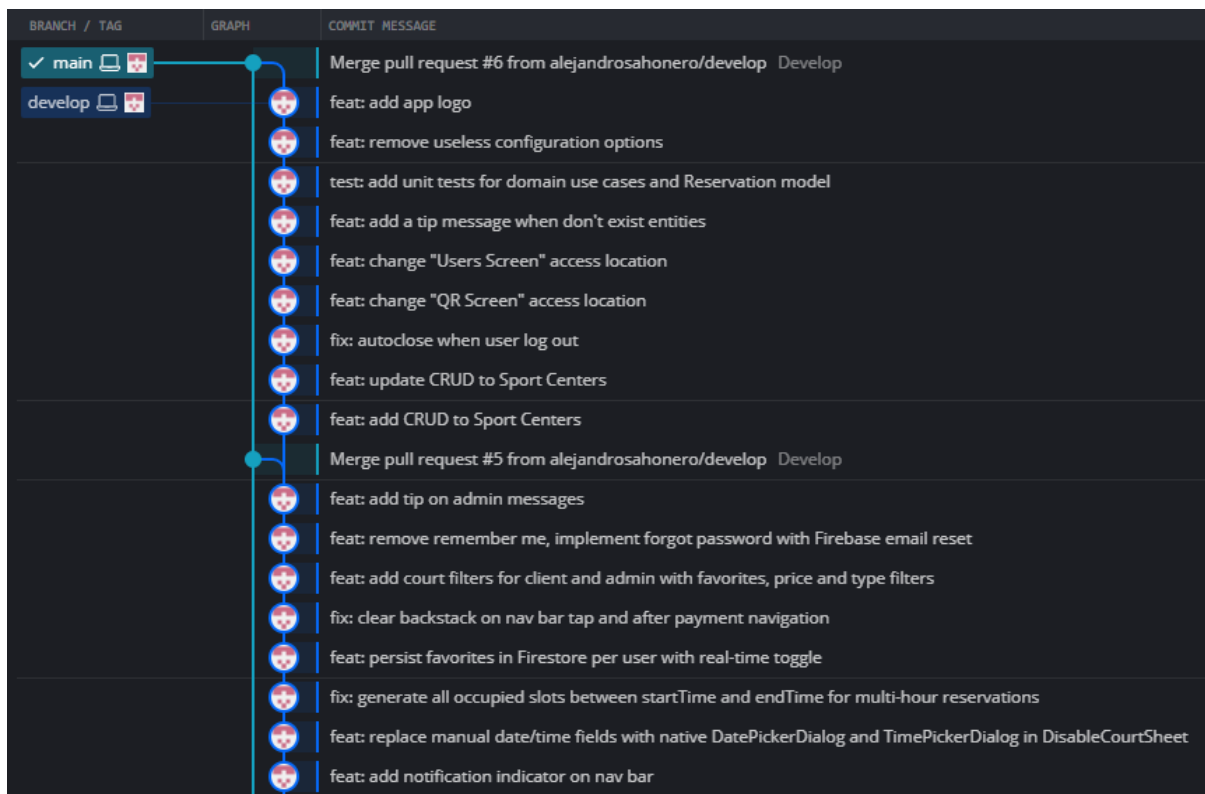


Ilustración 1: Gráfica del control de versiones (Git). Fuente: Elaboración propia

3.3. Cronograma y desarrollo de fases

El proyecto se ha completado en un periodo de 6 semanas, aunque el camino no ha estado exento de dificultades técnicas que han obligado a replantear ciertas decisiones iniciales:

3.3.1 Fase 1: Infraestructura base y desafíos de compatibilidad (Semanas 1-2)

La primera fase se centró en crear toda la estructura de paquetes y carpetas antes de escribir una sola línea de lógica. El objetivo era forzar la separación de capas (Dominio, Datos y UI)

desde el inicio para evitar problemas de mantenimiento futuros. El objetivo era montar los "cimientos" de la aplicación. Sin embargo, surgieron problemas importantes con las herramientas de construcción:

- **Conflicto con Android Gradle Plugin (AGP):** Al intentar usar las versiones más modernas, el sistema no funcionaba correctamente. El problema fue con la versión de KSP que no existía. La solución fue encontrar la versión correcta de KSP (2.2.10-2.0.2) y añadir el bloque de *sourceSets* que AGP 9.x requiere.
- **Configuración del entorno:** Se establecieron los paquetes, las dependencias iniciales y la conexión básica con Firebase. Fue una etapa arquitectónica donde la prioridad era tener un proyecto que compilara sin avisos ni fallos.

3.3.2 Fase 2: Diseño UI/UX y Prototipado (Semanas 2-3)

Esta fase se extendió más de lo previsto debido a la constante aparición de nuevas ideas para mejorar la experiencia del usuario. Se buscó un diseño limpio y moderno basado en Material Design 3, cuidando especialmente que el flujo entre pantallas fuera intuitivo tanto para el cliente como para el administrador. Aunque se usaron mockups en Figma como base, la implementación real en Android permitió mejorar mucho la experiencia de usuario.

- **Identidad visual:** Se diseñó una pantalla de inicio con una imagen de la ciudad de Logroño, lo que obligó a trabajar mucho la legibilidad. Se optó por una estructura donde la imagen ocupa el espacio sobrante sin superponerse al formulario de acceso.
- **Material Design 3 y Modo Oscuro:** Se implementó un tema oscuro nativo, definiendo "tokens" de diseño (como el rojo #E53935 para elementos principales). Esto asegura que toda la aplicación tenga una estética profesional y coherente en cualquier pantalla.
- **Superando el Figma:** La pantalla de selección de pistas y horarios se mejoró respecto al diseño original. Gracias a Jetpack Compose, se logró que la interfaz fuera reactiva: cuando el usuario selecciona una fecha, los horarios disponibles se actualizan al instante sin esperas.

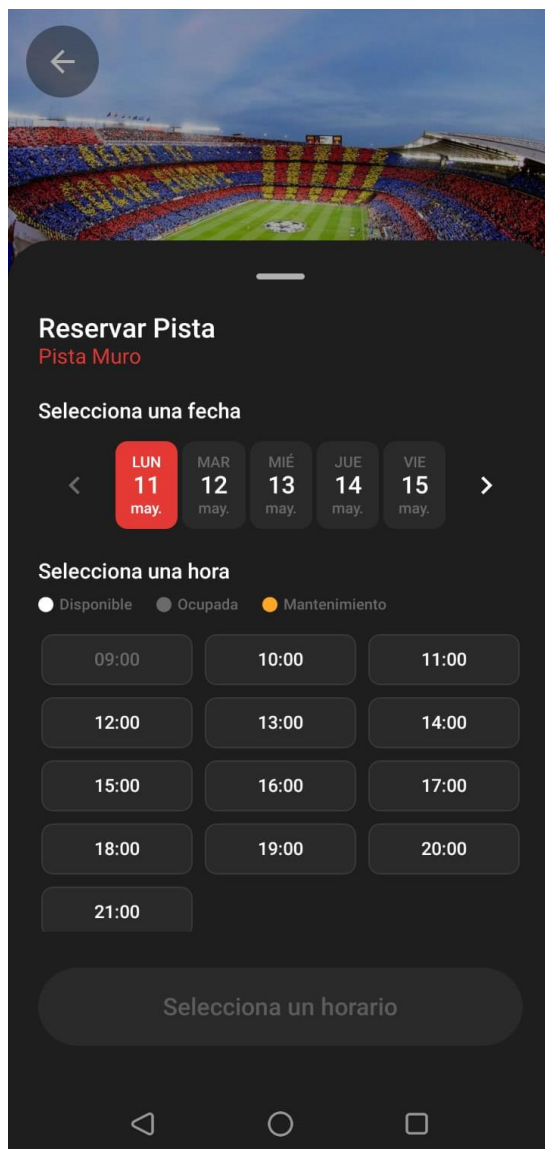
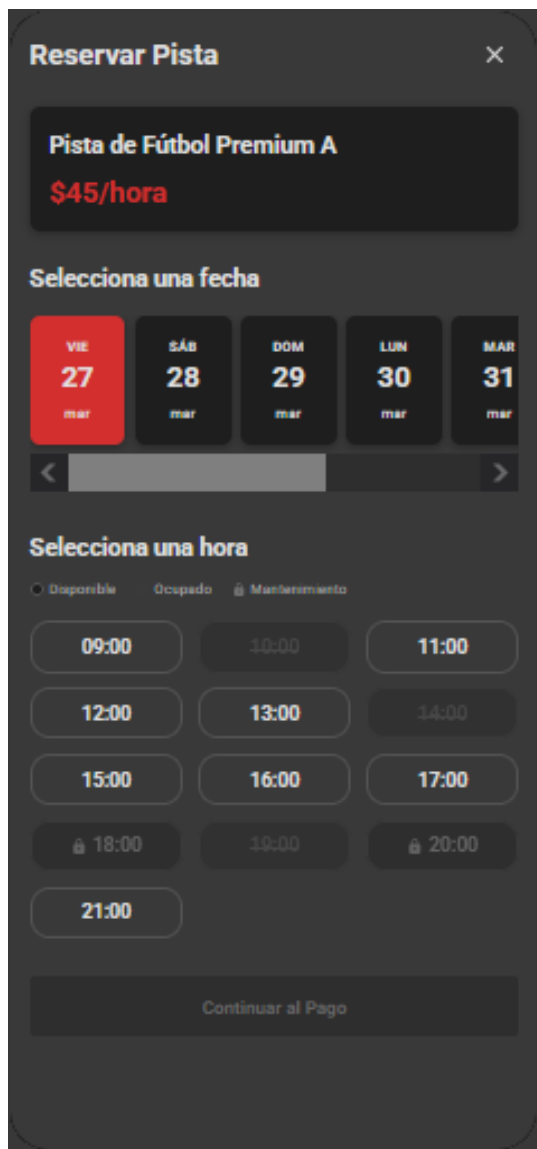


Ilustración 2: Comparativa Figma (Izquierda) vs Diseño final (Derecha). Fuente: Elaboración propia

3.3.3 Fase 3: Desarrollo de la lógica de datos (Semanas 3-4)

Aquí se tomó una de las decisiones más importantes del proyecto: abandonar Hilt (un sistema automático de gestión de piezas del código) para volver a un control manual.

- **El conflicto de Kotlin y Gradle:** Intenté usar Hilt, pero el sistema exigía bajar la versión de Kotlin para funcionar, algo que no quería hacer porque prefería usar las funciones más modernas del lenguaje.
- **AppContainer como solución:** En lugar de depender de una herramienta que reemplaza código pero que hace cosas “por detrás”, construí un *AppContainer* a mano. Esto significa que yo mismo controlo cómo se crea cada parte de la aplicación en el momento en que se inicia.

3.3.4 Fase 4: Funcionalidades Core y Automatización (Semanas 4-5)

Esta fue la etapa de desarrollo de la lógica más difícil. Aquí es donde CourtHub dejó de ser una aplicación de fotos de pistas para ser una herramienta de reservas real.

- **El problema de las reservas dobles:** Descubrí que dos personas podían reservar la misma pista a la vez. Aunque para el TFG el riesgo es bajo, se analizó el uso de transacciones en la base de datos para que el sistema "bloquee" el horario mientras alguien está pagando.
- **Reservas de varias horas (Multi-slot):** Un fallo importante fue que, al reservar por ejemplo de 10:00 a 12:00, el sistema solo marcaba como ocupada la primera hora. Tuve que corregir la lógica del repositorio para que el sistema "entienda" que una reserva ocupa todo el rango de horas intermedio.
- **Notificaciones y Sincronización:** Se implementó un historial para que el usuario pueda leer sus avisos uno por uno, guardándolos en Firestore para que no se pierdan al cerrar la sesión. También se usaron "*listeners*" en tiempo real para que, si el administrador cambia algo, el cliente lo vea sin tener que reiniciar la aplicación.

3.3.5 Fase 5: Depuración, pulido y documentación (Semana 6)

La última fase sirvió para pulir detalles que se habían pasado por alto y que son vitales para que la aplicación sea útil en la vida real.

- **Categorización por centros:** Me di cuenta de que, si un usuario tiene que buscar entre una lista infinita de pistas, se acabaría cansando. Añadí a última hora una organización por "centros deportivos", para que puedas elijas el sitio y luego también ver sus pistas.
- **Limpieza de la interfaz:** El administrador tenía demasiadas opciones en el menú inferior (Nav bar), lo cual es un error de diseño según las guías oficiales de Google, es por eso que moví los accesos menos usados al panel principal de estadísticas (KPIs), dejando un menú de solo 5 iconos, mucho más limpio y profesional.
- **Detalles de última hora:** Se incluyó el sistema de favoritos (para que el usuario guarde sus pistas preferidas), se añadieron filtros rápidos en las listas y se diseñó el logo definitivo, cerrando así el ciclo de desarrollo.

4. Diseño del sistema

En este capítulo se detalla la ingeniería detrás de CourtHub. Para que el sistema sea capaz de gestionar reservas en tiempo real y permitir el acceso autónomo de los usuarios, se ha diseñado una estructura interna que prioriza la estabilidad, la facilidad de mantenimiento y la seguridad de los datos.

4.1. Arquitectura de software

La base de CourtHub se apoya en la combinación de *Clean Architecture*³ y el patrón *MVVM*⁴(Model-View-ViewModel) estructurada en tres capas diferenciadas: Dominio, Datos y Presentación (UI) con una regla de dependencia estricta, las capas externas conocen a las internas, pero nunca al revés. Esta separación garantiza que el núcleo de la aplicación sea independiente de marcos de trabajo (*frameworks*⁵), bases de datos o interfaces externas. A diferencia de las arquitecturas tradicionales, aquí el flujo de dependencias siempre apunta hacia el centro, lo que permite que el negocio no se vea afectado si, por ejemplo, se decidiera cambiar Firebase por otra tecnología en el futuro.

4.1.1. Capa de dominio

La capa de dominio, que es el corazón de la aplicación, es la capa más interna y crítica del sistema. Su característica principal es que no importa ninguna clase de Android, Firebase ni Room⁶; es código Kotlin puro. Aquí es donde residen las reglas que definen qué es CourtHub y cómo funciona, independientemente de la tecnología de persistencia.

- **Entidades de dominio:** Representan los objetos de negocio, como *Court* o *Reservation*.
- **Use Cases (Casos de Uso):** Son los encargados de orquestar la lógica de negocio. Cada acción específica del usuario (como realizar o cancelar una reserva) se encapsula en un caso de uso que actúa como el único punto de entrada a esa funcionalidad.

³ Patrón de arquitectura que busca la separación de responsabilidades en capas, permitiendo que la lógica de negocio sea independiente de las bases de datos y de la interfaz visual.

⁴ Patrón de diseño que facilita la separación entre la lógica de la interfaz de usuario y la lógica de negocio, utilizando un mediador (ViewModel) para gestionar el estado de la pantalla.

⁵ Estructura de herramientas y reglas ya preparadas que facilita y acelera el desarrollo de aplicaciones, porque claro, reinventar la rueda cada martes no era suficiente castigo para los programadores.

⁶ Es una biblioteca y su objetivo es simplificar el acceso a bases de datos locales en aplicaciones Android

Un ejemplo real de esta independencia se observa en el proceso de cancelación. El siguiente fragmento muestra cómo un *Use Case* coordina la lógica sin conocer detalles técnicos de la base de datos:

```
class CancelReservationUseCase(  
    private val repository: IReservationRepository  
) {  
    suspend operator fun invoke(reservation: Reservation): Result<Unit>  
    {  
        if (!reservation.canBeCancelled())  
            return Result.failure(  
                IllegalStateException("No se puede cancelar con menos de  
24h")  
            )  
        return repository.cancelReservation(reservation.id, "Cancelado  
por el usuario")  
    }  
}
```

Ilustración 3: Ejemplo de un "Use Case" escrito en el lenguaje de programación kotlin: Fuente: Elaboración propia

Es fundamental destacar que la lógica de validación temporal reside en la función *canBeCancelled()* del propio modelo *Reservation*. Al ubicar esta regla en la entidad de dominio y no en la interfaz de usuario ni en el ViewModel, aseguramos que la política de cancelaciones sea consistente en toda la aplicación y fácilmente testeable.

4.1.2. Capa de datos

Para mantener la pureza de la capa de dominio, la capa de datos utiliza modelos específicos para cada fuente de información y Mappers para transformar estos datos en entidades de negocio. Esta separación evita que las anotaciones de Room o los campos específicos de Firestore "contaminen" el dominio.

- **Court.kt (Dominio):** Modelo puro usado por la lógica de negocio y la UI. No conoce bases de datos.
- **CourtDto.kt (Data Transfer Object):** Estructura optimizada para la comunicación con Firestore, preparada para la serialización y desnormalización de datos en la nube.
- **CourtEntity.kt (Entity):** Modelo con anotaciones específicas de Room para la persistencia local y la gestión de claves primarias⁷ en el dispositivo.
- **Mappers:** Funciones de extensión que convierten, por ejemplo, un *CourtDto* en un *Court* al descargar datos, o un *Court* en un *CourtEntity* al guardarlo para el modo offline.

⁷ Campo (o conjunto de campos) que identifica de forma única cada registro de una tabla en una base de datos y no puede repetirse ni ser nulo.

4.1.3. Capa de Presentación

La capa de UI se comunica con el dominio únicamente a través de los Use Cases, utilizando el patrón MVVM para gestionar el estado de la pantalla. Para ensamblar todas estas piezas sin depender de herramientas automáticas que oculten la lógica, se ha implementado un sistema de Inyección de Dependencias manual. A través de un AppContainer, se instancian los repositorios e interfaces, garantizando que cada componente reciba exactamente lo que necesita para funcionar, lo que facilita el mantenimiento y la escalabilidad del sistema.

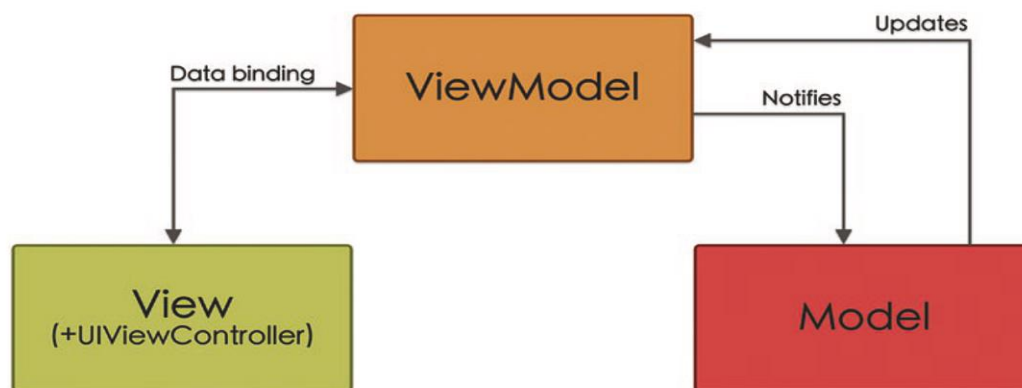


Ilustración 4: Arquitectura MVVM. Fuente: medium.com

Para ilustrar cómo funciona esta separación en la práctica, la siguiente tabla resume los casos de uso implementados en la capa de dominio:

Tabla 1: Casos de usos implementados

Use Case	Qué valida antes de actuar	Qué devuelve
LoginUseCase	Email y contraseña no vacíos	<i>Result<User></i>
RegisterUseCase	Contraseñas coinciden, mínimo 6 caracteres	<i>Result<User></i>
CancelReservationUseCase	Más de 24h de antelación, reserva confirmada	<i>Result<Unit></i>
CreateReservationUseCase	Datos de reserva completos	<i>Result<String></i> (ID)

GetAvailableSlotsUseCase	-	<i>List<TimeSlot></i> con estado
ValidateAccessCodeUseCase	Código VALID, dentro del horario exacto	ValidationResult
DisableCourtUseCase	Motivo no vacío, fecha inicio < fecha fin	<i>Result<Unit></i>
GenerateAccessCodeUseCase	-	OTP de 6 dígitos aleatorio

Cada use case tiene una sola responsabilidad y valida las reglas de negocio antes de llamar al repositorio. Si la validación falla, el repositorio nunca llega a ejecutarse.

4.2 Stack tecnológico

Para construir CourtHub se ha seleccionado un conjunto de herramientas modernas que permitan que la aplicación sea rápida, segura y fácil de mantener. A continuación, se detalla el stack tecnológico completo y la razón por la que cada pieza es fundamental para el proyecto:

- **Lenguaje de programación (Kotlin 2.2.10):** Se ha utilizado Kotlin por ser el lenguaje oficial para el desarrollo en Android. Sus funciones de seguridad, como el control de nulos (*null safety*), evitan gran parte de los errores que hacen que las aplicaciones se cierren solas. Además, permite usar *corrutinas*, que son las que hacen que la aplicación pueda descargar datos de internet en segundo plano sin que la pantalla se congele.
- **Interfaz de usuario (Jetpack Compose y Material Design 3):** En lugar de usar el sistema antiguo de vistas, se ha usado Compose, un kit moderno que permite crear la interfaz mediante código declarativo. Esto facilita mucho que la pantalla se actualice sola cuando cambian los datos de una reserva. Todo el aspecto visual sigue las reglas de Material Design 3, lo que le da un acabado profesional y coherente.
- **Base de datos remota (Firebase Firestore):** Es el corazón de la nube de CourtHub. Se ha elegido porque permite que los datos se sincronicen en tiempo real entre el administrador y el cliente. Además, tiene una función de persistencia que permite que parte de la información siga disponible, aunque el usuario pierda la conexión un momento.
- **Base de datos local (Room 2.7.1):** Aunque casi todo está en la nube, se ha implementado Room para guardar los datos críticos en la memoria del teléfono. Esto

es clave para que los códigos QR y de acceso funcionen siempre, incluso si el usuario está en la puerta del centro y no tiene internet. Su elección es una respuesta directa a las condiciones de baja señal en interiores, permite que el sistema de entrada funcione con latencia cero, sin importar las interferencias o muros que bloqueen la comunicación con el servidor.

- **Autenticación (Firebase Auth y Credential Manager API):** Para que el inicio de sesión sea seguro y profesional, se ha usado Firebase Auth junto con la nueva API de Google. Esto permite que el usuario entre con su cuenta de Google de forma rápida y con un sistema de seguridad de nivel bancario.
- **Navegación (Navigation Compose 2.9.0):** Se encarga de gestionar el paso de una pantalla a otra de forma segura. Permite que, al pasar de la lista de pistas al pago, todos los datos de la reserva (hora, precio, ID de la pista) lleguen correctamente sin perderse por el camino.
- **Gestión de tareas (WorkManager):** Se ha usado para programar los recordatorios de las reservas. WorkManager asegura que el teléfono envíe la notificación al usuario a la hora exacta, incluso si la aplicación está cerrada o el móvil está en modo ahorro de batería.
- **Generación y lectura de códigos (ZXing y CameraX + ML Kit):** La librería ZXing genera el dibujo del código QR con los datos de la reserva. Para leerlo desde el móvil del administrador, se ha usado CameraX para controlar la cámara y ML Kit de Google para que el reconocimiento del código sea instantáneo y preciso.
- **Inyección de dependencias (AppContainer):** Como se explicó anteriormente, en lugar de usar librerías automáticas que ocultan el funcionamiento del código, se ha creado un contenedor manual. Esto permite que la arquitectura de la aplicación sea transparente y que sea mucho más fácil encontrar fallos en la comunicación entre las distintas partes del sistema.

4.3. Modelo de datos y persistencia

El sistema de almacenamiento de CourtHub se ha diseñado pensando tanto en el rendimiento como en la experiencia del usuario. Para ello, se ha utilizado una combinación entre Firebase Firestore como base de datos en la nube y Room como sistema de almacenamiento local dentro del dispositivo Android.

A diferencia de bases de datos tradicionales como MySQL, Firestore funciona mediante documentos y colecciones en lugar de tablas relacionadas entre sí. Esto obliga a organizar la información de una forma distinta. En algunos casos, se decidió repetir ciertos datos dentro de una reserva, como el nombre del usuario o de la pista. Aunque a primera vista pueda

parecer innecesario, esta decisión mejora mucho la velocidad de carga y evita realizar múltiples consultas seguidas a la base de datos.

Por ejemplo, si una reserva solo guardara el identificador de la pista (*courtId*), cada vez que el administrador quisiera ver una lista de reservas sería necesario buscar también la información de cada pista por separado. Si existieran 50 reservas, habría que realizar decenas de consultas adicionales únicamente para mostrar los nombres correspondientes. Para evitar este problema, cada reserva almacena directamente información ya preparada para mostrarse en pantalla.



Ilustración 5: Diagrama de datos de Firebase

Entre los campos desnormalizados más importantes se encuentran:

- **userName:** nombre del usuario que realizó la reserva.
- **courtName:** nombre de la pista reservada.
- **centerName:** nombre del centro deportivo al que pertenece la pista.

Gracias a esto, el sistema puede mostrar toda la información necesaria con una sola lectura, haciendo que la aplicación responda de forma más rápida y reduciendo el consumo de red.

Por otro lado, la base de datos local utiliza un enfoque más cercano al modelo relacional tradicional. En Room sí se aprovechan relaciones entre tablas y claves foráneas para mantener una estructura más limpia y optimizada dentro del dispositivo.

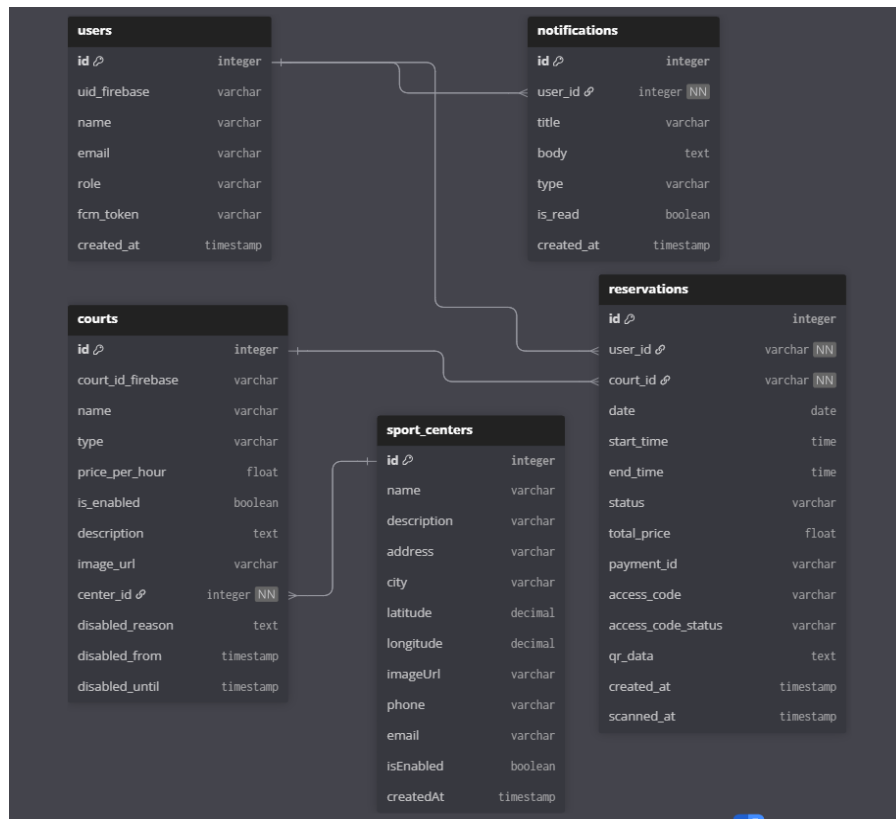


Ilustración 6: Diagrama de datos local

Para poder sincronizar correctamente Firestore con Room, fue necesario adaptar algunos aspectos técnicos entre ambos sistemas:

- **Identificadores y Claves Primarias:** En SQLite se utilizan enteros autoincrementales (*id*) como claves primarias⁸ para optimizar la indexación⁹ y las relaciones internas. Como los documentos de Firestore utilizan cadenas de texto únicas (*UIDs*¹⁰), es necesario crear campos específicos como *uid_firebase* o *court_id_firebase* para mantener el vínculo de sincronización entre ambas bases de datos sin comprometer la velocidad de las consultas locales.
- **Convenciones de Nomenclatura:** Mientras que en entornos NoSQL y código Kotlin/Java predomina el camelCase¹¹, el estándar en bases de datos SQL es el snake_case¹².

⁸ Identificador único que se genera y aumenta automáticamente al insertar un nuevo registro en una tabla de base de datos.

⁹ Proceso de crear estructuras (índices) que permiten localizar y acceder a los datos más rápido sin tener que recorrer toda la tabla.

¹⁰ (Unique Identifier) es un identificador único que se asigna a un usuario, registro o entidad para distinguirlo de cualquier otro dentro de un sistema.

¹¹ Convención de escritura donde las palabras se unen sin espacios y cada palabra (excepto la primera, en el caso típico) comienza con mayúscula, como: *miVariableEjemplo*.

¹² convención de escritura donde las palabras se escriben en minúscula y separadas por guiones bajos, como: *mi_variable_ejemplo*.

- **Normalización vs. Desnormalización:** En Firestore se desnormalizan datos (como incluir `userName` dentro de una reserva) para evitar consultas múltiples, ya que NoSQL no soporta JOINS¹³ de forma eficiente. En SQLite, al ser una base de datos relacional, se eliminan esos campos redundantes y se utilizan claves foráneas (`user_id`, `court_id`). Esto permite obtener la información actualizada mediante una única consulta con unión de tablas, reduciendo el tamaño de la base de datos local.
- **Tipado Estricto:** SQLite requiere la definición explícita de tipos de datos (*Integer*, *Varchar*, *Float*, *Boolean*). Los nombres de los campos a menudo se adaptan para reflejar su contenido técnico o restricciones, como el uso de sufijos `_id` para dejar clara la naturaleza de la relación entre tablas.

La siguiente tabla resume cómo se distribuyen los datos entre el almacenamiento local y la nube:

Tabla 2: Comparación de responsabilidades en la persistencia

Tipo de dato	Almacenamiento	Razón Técnica
Perfil de usuario y roles	Firestore / Room	Acceso rápido y seguridad de roles.
Pistas y Centros	Firestore	Actualización constante de disponibilidad.
Reservas confirmadas	Firestore / Room	Disponibilidad del código QR sin conexión.
Notificaciones e Historial	Firestore	Registro persistente entre dispositivos.

Para finalizar, el sistema de acceso utiliza códigos QR generados a partir de información única de cada reserva. Estos códigos contienen datos relacionados con la pista, la fecha y el acceso del usuario. Posteriormente, la librería ZXing se encarga de transformar esa información en un código QR que puede ser leído rápidamente desde el sistema de validación de entrada.

5. Implementación y configuración

Una vez definida la arquitectura y el stack tecnológico, la fase de implementación se centró en construir un sistema robusto que garantizará tanto la seguridad de los datos como la fiabilidad en el proceso de reserva y acceso. En esta sección se describen los aspectos críticos de la configuración del servidor, el desarrollo del módulo de escaneo y la lógica aplicada para resolver los errores técnicos detectados durante el desarrollo.

¹³ Operaciones que combinan filas de dos o más tablas usando una condición común entre ellas (normalmente claves relacionadas) para poder consultar datos relacionados como si estuvieran en una sola tabla.

5.1. Seguridad y Autorización del servidor

La seguridad en CourtHub no se limita a la interfaz de la aplicación, sino que se establece en el servidor mediante las reglas de seguridad de Firestore (Security Rules). El principio fundamental aplicado es el de "denegación por defecto", lo que significa que el servidor bloquea cualquier intento de lectura o escritura a menos que exista una regla específica que lo permita explícitamente.

Para proteger la privacidad de los usuarios, se han implementado reglas que aseguran que un cliente solo pueda acceder a sus propias reservas y datos personales. Por ejemplo, al intentar consultar una reserva, el servidor comprueba que el identificador del usuario autenticado coincida con el campo *userId* almacenado en el documento de la reserva. De este modo, aunque se intente manipular la aplicación desde el exterior, los datos de otros usuarios permanecen inaccesibles. Estas reglas actúan como una capa de seguridad independiente de la aplicación que no depende del código instalado en el teléfono del usuario, sino de la validación directa en la infraestructura de Google.

A continuación, se detalla la implementación técnica de las reglas para la colección de reservas, que constituye la parte principal del sistema de reservas:

```
match /reservations/{reservationId} {  
  // El cliente solo lee SUS reservas  
  allow read: if request.auth != null &&  
               resource.data.userId == request.auth.uid;  
  // El admin lee todas  
  allow read: if isAdmin();  
  // Al crear, el userId DEBE ser el del usuario autenticado  
  allow create: if request.auth != null &&  
                  request.resource.data.userId == request.auth.uid &&  
                  request.resource.data.status == "confirmed";  
}
```

Ilustración 7: Security Rules de Firestore

Es fundamental comprender que estas reglas se evalúan en los servidores de Google, no en el código de la aplicación. Esto significa que, aunque un atacante lograra "hackear" la aplicación o interceptar las llamadas a la API, los datos de otros usuarios seguirían siendo totalmente inaccesibles, ya que el servidor valida la identidad del solicitante (*request.auth.uid*) contra los datos protegidos (*resource.data.userId*) antes de liberar cualquier información.

5.1.1. Protección de integridad de perfiles y roles

Un aspecto crítico en cualquier sistema con privilegios es evitar la escalada de privilegios, es decir, que un usuario malintencionado intente cambiar su rol de "cliente" a "administrador".

En CourtHub, esto se ha resuelto mediante una lógica de escritura restringida en la colección de usuarios.

Aunque el usuario tiene permiso para actualizar sus datos personales (como su nombre o foto de perfil), el campo *role* está bloqueado ante cualquier intento de modificación por parte del cliente. El servidor verifica que el estado final del documento no contenga cambios en los campos sensibles, asegurando que el acceso a las funciones de administración (como el borrado de pistas o la gestión de KPIs) permanezca restringido exclusivamente a las cuentas autorizadas previamente en la base de datos.

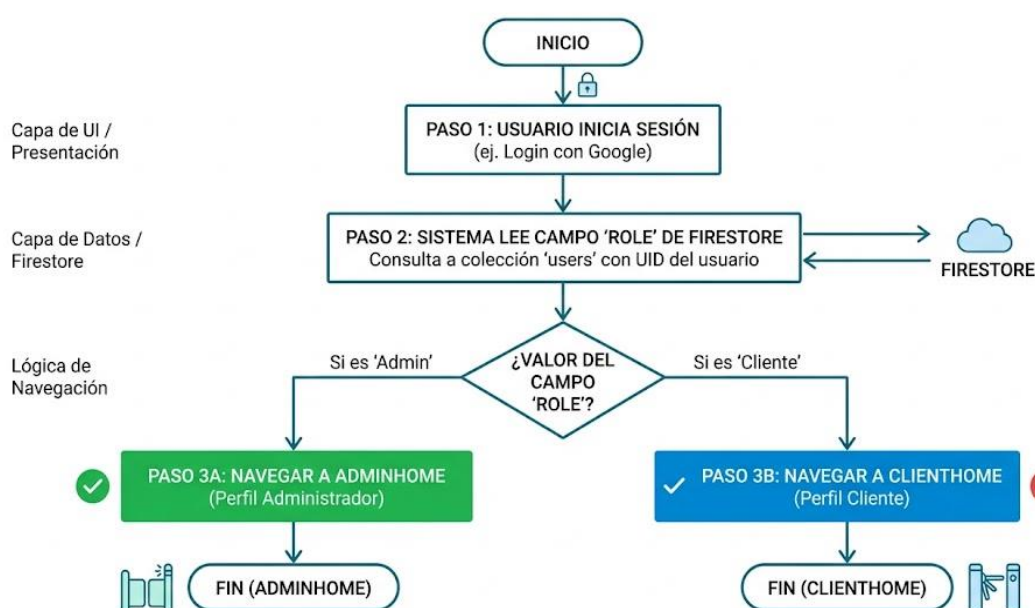


Ilustración 8: Flujo del sistema de roles

5.2. Desarrollo del Sistema de Acceso

El núcleo de la automatización en CourtHub es el sistema de validación de accesos, diseñado para sustituir la necesidad de una intervención humana constante en la entrada del centro deportivo. Este módulo utiliza la librería CameraX para gestionar el flujo de la cámara y ML Kit para el procesamiento de imágenes en tiempo real.

Durante el escaneo, la aplicación analiza los cuadros de imagen (frames) buscando un código QR que contenga la información de una reserva válida. Cuando el sistema detecta y valida el código, la pantalla muestra un mensaje visual en verde, lo que en una instalación real enviaría una señal para desbloquear las puertas del centro. Si el escaneo falla o la reserva no es válida para ese horario, el sistema mantiene el acceso bloqueado. Como medida de seguridad adicional y para garantizar la accesibilidad, se ha implementado el código OTP como una alternativa crítica. Este sistema es fundamental en situaciones donde el hardware de escaneo

pueda presentar fallos, la lente de la cámara esté sucia o simplemente el centro deportivo aún no haya instalado un lector físico de QR. Al introducir manualmente este código de seis dígitos, el usuario puede acceder a las pistas de la misma manera que con el escaneo, asegurando que el usuario pueda entrar incluso si falla el lector QR.

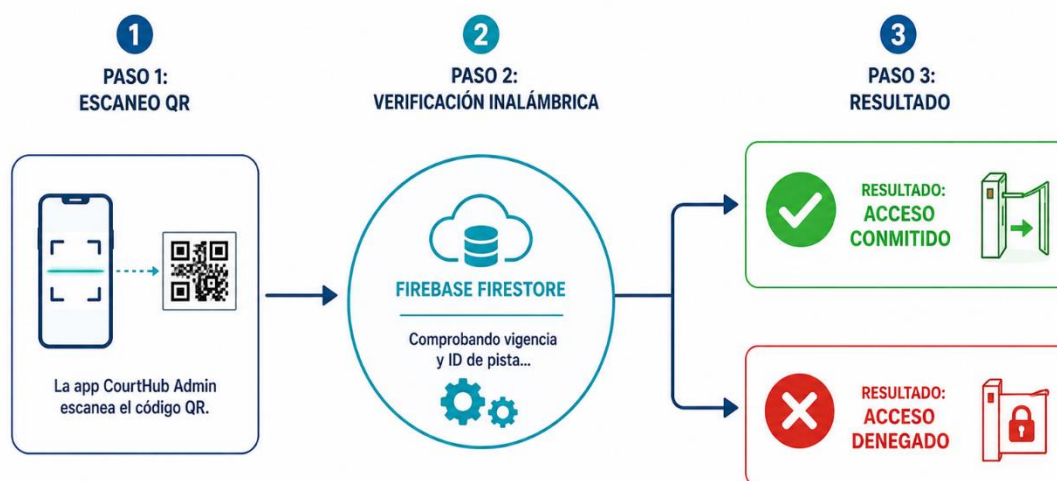


Ilustración 9: Diagrama de flujo del proceso de validación de acceso

5.3. Resolución de problemas y lógica de negocio crítica

El desarrollo de la lógica de reservas presentó varios desafíos técnicos que requirieron soluciones específicas para garantizar la integridad del sistema.

- **Gestión de Concurrencia (Double-Booking):** Uno de los hallazgos más críticos durante la fase de pruebas fue el riesgo de concurrencia o double-booking. Se detectó que, en un escenario de uso real con múltiples usuarios simultáneos, existía la posibilidad de que dos personas reservaran la misma pista en el mismo horario. Tras analizar este problema, se estudió la implementación de transacciones de base de datos y el uso de documentos de bloqueo (locks) por cada franja horaria. Sin embargo, para este prototipo se decidió mantener la lógica funcional base, documentando la implementación de transacciones robustas como una mejora obligatoria para la fase de producción, donde la integridad atómica de las operaciones es innegable.
- **Reservas Multi-slot:** Inicialmente, el sistema solo registraba la hora de inicio de una reserva, dejando los huecos siguientes como disponibles. Se corrigió la lógica en el repositorio para que, al realizar una reserva de varias horas, el sistema identifique y bloquee todo el rango horario ocupado.

- **Actualizaciones del Administrador en Tiempo Real:** Se detectó que los cambios realizados por el administrador, como deshabilitar una pista, no se reflejaban de inmediato en la aplicación del cliente sin reiniciarla. La solución fue sustituir las consultas puntuales por "escuchadores" (listeners) de Firestore, que actualizan la interfaz de forma automática y reactiva en cuanto ocurre un cambio en la base de datos. Técnicamente, este mecanismo se implementa mediante el patrón *callbackFlow* de Kotlin. Firestore usa listeners de Java que no son compatibles directamente con el sistema de Flow moderno de Kotlin. *callbackFlow* actúa como puente entre ambos mundos: cuando Firestore detecta un cambio en una colección, el listener recibe el dato nuevo y lo emite al Flow, que lo propaga automáticamente al ViewModel y de ahí a la pantalla, sin que el usuario tenga que pulsar ningún botón ni recargar nada:

```
override fun getCourts(): Flow<List<Court>> = callbackFlow {
    val listener = firestore.collection("courts")
        .addSnapshotListener { snapshot, error ->
            if (error != null) { close(error);
return@addSnapshotListener }
            val courts = snapshot?.documents?.mapNotNull { doc ->
doc.toObject(CourtDto::class.java)?.toDomain(doc.id)
            } ?: emptyList()
            trySend(courts) // emite al Flow cada vez que
Firestore cambia
        }
    awaitClose { listener.remove() } // limpia el listener al
cerrar
}
```

Ilustración 10: Ejemplo del repositorio. Fuente: Elaboración propia.

La clave es *awaitClose*: cuando el ViewModel deja de observar el Flow (por ejemplo, porque el usuario cierra la pantalla), el listener de Firestore se elimina automáticamente, evitando fugas de memoria.

- **Historial de Notificaciones:** Para asegurar que los avisos de reserva no se perdieran, se pasó de un sistema de notificaciones locales a uno basado en el servidor. Ahora, cada notificación se guarda en el historial de Firestore, permitiendo que el usuario pueda consultarlas de manera individual y persistente desde cualquier dispositivo.

6. Pruebas y control de calidad (QA)

El proceso de verificación de CourtHub está diseñado para garantizar que todas las funciones críticas, especialmente aquellas relacionadas con el acceso y la persistencia de datos, funcionen correctamente en diferentes escenarios.

Dado que el proyecto es complejo, me he centrado en pruebas de campo y de integración. Quería asegurarme de que la comunicación entre el dispositivo del cliente y el del administrador sea fluida y segura.

Para lograr esto, he seguido una estrategia de pruebas en pirámide. Empecé validando la lógica interna con pruebas unitarias. Luego, terminé con pruebas de integración en dispositivos reales.

Esto me permitió asegurarme de que todo funcione correctamente.

6.1. Pruebas Unitarias

La base de la estrategia de calidad de CourtHub reside en las pruebas unitarias, centradas exclusivamente en la capa de dominio. Esta capa es la única que contiene lógica de negocio pura, sin dependencias de marcos de trabajo externos como Android, Firebase o Room. Al aislar esta lógica, es posible ejecutar los tests en la máquina local (JVM) en milisegundos, permitiendo un ciclo de desarrollo rápido y fiable. Se han implementado baterías de pruebas para los componentes más críticos del sistema utilizando JUnit 4 y MockK para la creación de dobles de prueba:

- **Validación de cancelaciones:** Se ha testeado exhaustivamente la regla de negocio que permite cancelar reservas solo con más de 24 horas de antelación. Se han cubierto todos los escenarios posibles (happy path¹⁴, límites temporales y estados de reserva inválidos) para evitar errores que pudieran tener un impacto económico directo en el centro deportivo.
- **Seguridad en el registro y acceso:** Estos tests verifican que el sistema actúa como "guardián", validando los datos antes de enviarlos al servidor. Se comprueba que el sistema rechace correos mal formados o contraseñas débiles, asegurando una experiencia de usuario sin errores silenciosos.
- **Lógica de disponibilidad:** Al ser la función más compleja, se ha verificado que el filtrado de horarios ocupados funcione incluso en casos límite, como cuando todos los slots de una pista están llenos.

¹⁴ Se refiere al recorrido que hace un usuario por la aplicación cuando todo sale perfecto: rellena bien los datos, tiene internet y no hay errores de servidor. Es el escenario "feliz" donde el sistema no encuentra ningún obstáculo.

- **Gestión de corrutinas:** Dado que la aplicación es asíncrona, se ha utilizado *runTest* para controlar el paso del tiempo en las pruebas y asegurar que los flujos de datos (Flows¹⁵) se comporten de manera predecible.

```
@Test
fun `canBeCancelled returns true when more than 24 hours remain`()
{
    val reservation = makeReservation(hoursFromNow = 26) // Usamos
26 para evitar fallos por truncamiento de minutos
    assertTrue(reservation.canBeCancelled())
}

@Test
fun `canBeCancelled returns false when less than 24 hours remain`()
{
    val reservation = makeReservation(hoursFromNow = 23)
    assertFalse(reservation.canBeCancelled())
}

@Test
fun `canBeCancelled returns false when reservation is already
cancelled`() {
    val reservation = makeReservation(hoursFromNow = 26)
        .copy(status = ReservationStatus.CANCELLED)
    assertFalse(reservation.canBeCancelled())
}

@Test
fun `canBeCancelled returns false when reservation is expired`() {
    val reservation = makeReservation(hoursFromNow = 26)
        .copy(status = ReservationStatus.EXPIRED)
    assertFalse(reservation.canBeCancelled())
}
```

Ilustración 11: Tests realizados en Android Studio

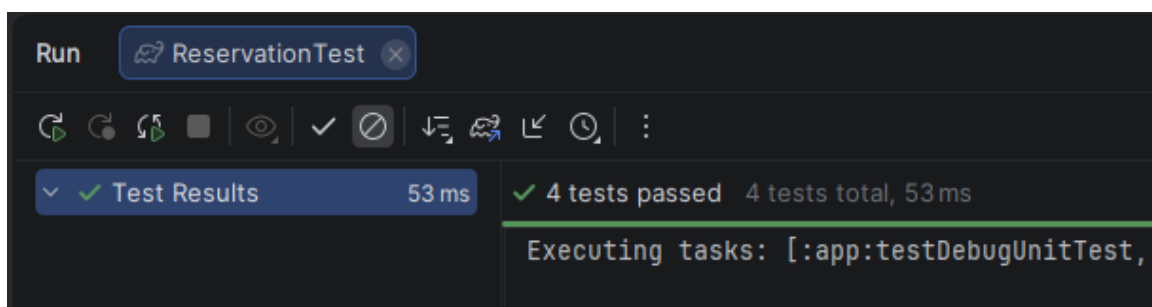


Ilustración 12: Captura del panel de resultados de los tests de la ilustración anterior. Fuentes: Elaboración propia

¹⁵ Es la tecnología de Kotlin que permite que la información "fluya" en tiempo real. Si el administrador cambia el precio de una pista, el "Flow" detecta ese cambio y actualiza la pantalla del usuario al instante sin que este tenga que pulsar nada.

6.2 Entorno de Pruebas

Complementando a los tests unitarios, se han realizado verificaciones sobre un dispositivo Android físico. Esta decisión no solo responde a las limitaciones técnicas del equipo de desarrollo, sino que representa una ventaja estratégica: permite evaluar el rendimiento real de la cámara durante el escaneo, la velocidad de respuesta de la interfaz y el comportamiento de la batería. Probar la aplicación en un terminal real ha facilitado la detección de "bugs" visuales y de rendimiento que a menudo pasan desapercibidos en entornos virtuales, garantizando que el software sea robusto en manos del usuario final.

6.3 Verificación de la Persistencia y Acceso Offline

Uno de los pilares de CourtHub es su capacidad para funcionar en entornos con baja conectividad, habituales en las instalaciones deportivas. Para validar esta característica, se realizó una prueba de estrés desconectando totalmente la señal Wi-Fi y de datos móviles del dispositivo tras haber iniciado sesión.

El resultado confirmó la eficacia de la arquitectura híbrida: al haber iniciado sesión previamente para realizar la reserva, el sistema mantiene la sesión activa y permite al usuario navegar por sus pistas favoritas y, lo más importante, acceder a sus códigos QR de reserva. Gracias a la base de datos local de Room, la información crítica necesaria para entrar al centro permanece disponible en la memoria del teléfono. Esta prueba demuestra que el sistema es capaz de superar los "puntos ciegos" de cobertura sin interrumpir el servicio al cliente.

6.4 Pruebas de Integración y Validación de Acceso

La prueba definitiva del sistema consistió en la simulación completa del ciclo de reserva y acceso. Para ello, se generó un código QR real desde el perfil de un usuario tras completar una reserva en el sistema. Posteriormente, utilizando la función de escaneo implementada en el rol de administrador, se procedió a la lectura del código generado por la propia aplicación.

La integración entre la lógica de generación (ZXing) y la de lectura (CameraX + ML Kit) funcionó correctamente, logrando una validación instantánea. El sistema reconoció los datos cifrados en el QR, verificó la vigencia de la reserva en la base de datos y mostró la confirmación de acceso permitida. Esta prueba de "extremo a extremo" valida que CourtHub cumple su promesa técnica: un sistema de acceso autónomo, seguro y funcional que elimina la necesidad de una supervisión manual constante.

7. Ampliaciones y posibles mejoras

CourtHub ha sido diseñado como un Producto Mínimo Viable (MVP) totalmente funcional, pero con una arquitectura que permite su expansión sin tener que reescribir el sistema desde cero. En esta sección se detallan las líneas de mejora que harían pasar a la aplicación de un prototipo avanzado a una solución comercial de alto nivel.

7.1 Integración real de pasarela de pagos

Actualmente, el sistema utiliza un entorno de pruebas (Sandbox¹⁶) para simular los pagos de las reservas. Una mejora crítica para el despliegue real sería la integración del SDK de Stripe en modo producción. Esto requeriría desarrollar un "backend" intermedio (como una Cloud Function) para gestionar las claves secretas de forma segura, ya que estas nunca deben estar dentro de la aplicación móvil para evitar robos de datos. Además, esto permitiría implementar confirmaciones de pago automáticas mediante "webhooks", que son avisos que el banco envía directamente a nuestro servidor para confirmar que el dinero ha llegado correctamente.

7.2 Escalabilidad y modelo Multi-tenant

Aunque CourtHub ya permite gestionar varios centros deportivos, el sistema actual está pensado para una sola organización que sea dueña de todos esos centros. Una mejora lógica sería convertir la aplicación en una plataforma "multi-tenant"¹⁷ o de multi-propiedad. Esto significaría que diferentes empresas independientes podrían registrarse y gestionar sus propios recintos dentro de la misma aplicación, manteniendo sus datos y ganancias totalmente separados. Gracias al uso de Clean Architecture, este cambio solo afectaría a las reglas de seguridad y a la base de datos, sin tener que tocar el diseño o la lógica de las pantallas.

7.3 Internacionalización y accesibilidad técnica

En la versión actual, todos los textos y mensajes de error están escritos directamente en español. Para que la aplicación pueda escalarse a otros mercados, es necesario implementar

¹⁶ Un entorno de pruebas seguro donde puedes simular pagos o acciones reales sin usar dinero de verdad ni romper nada en el sistema oficial.

¹⁷ Arquitectura de software donde una única instancia de la aplicación sirve a múltiples clientes (inquilinos), manteniendo los datos de cada uno aislados y privados.

la internacionalización. Esto consiste en sacar todos los textos a archivos de recursos específicos para que la aplicación cambie de idioma automáticamente según la configuración del teléfono del usuario. Dentro de esta mejora, se incluiría un manejo de errores más profesional. En lugar de mostrar un mensaje genérico cuando algo falla, el sistema debería detectar si el problema es de conexión, de servidor o de permisos, y explicarle al usuario qué está pasando exactamente y cómo puede solucionarlo.

7.4 Automatización total del ciclo de vida (CI/CD)

Para un equipo de desarrollo, una mejora sustancial sería implementar un flujo de CI/CD (Integración y Despliegue Continuos). Esto permitiría que, cada vez que se suba una mejora a GitHub, un sistema automático compile la aplicación, pase todos los tests unitarios y genere una versión lista para instalar. Esto asegura que nunca se publique una versión con errores graves de forma accidental.

8. Conclusión

El desarrollo de CourtHub ha permitido demostrar que la automatización de procesos en centros deportivos no es solo una cuestión de comodidad, sino una solución viable para mejorar la rentabilidad de los negocios locales. A continuación, se presenta un análisis del cierre del proyecto.

8.1 Análisis de resultados frente a objetivos iniciales

Tras completar el ciclo de desarrollo, se puede afirmar que se han cumplido los objetivos principales marcados al inicio del proyecto:

- **Automatización operativa:** Se ha logrado implementar un sistema de acceso autónomo mediante códigos QR y OTP que funciona en tiempo real. La simulación del escáner en el perfil de administrador confirma que el centro podría operar sin personal físico constante en la entrada.
- **Gestión por roles:** La aplicación diferencia correctamente entre el cliente y el administrador, permitiendo a este último una supervisión total de las pistas, centros y usuarios desde su dispositivo móvil, cumpliendo con el requisito de movilidad y control total.

- **Integridad de datos:** A pesar de los desafíos de concurrencia detectados, la lógica de negocio actual permite que las reservas se procesen de forma segura y que la disponibilidad de las pistas se actualice instantáneamente para todos los usuarios.

8.2 Valoración personal y aprendizaje técnico

Desde un punto de vista técnico, el mayor aprendizaje ha sido la implementación de una arquitectura profesional como Clean Architecture. Aunque al principio supuso una curva de aprendizaje pronunciada y obligó a descartar herramientas automáticas como Hilt para mantener el control total del código, el resultado es un software mucho más fácil de entender, testear y ampliar. El manejo de bases de datos NoSQL¹⁸ con Firebase ha cambiado la forma de entender la estructura de la información, pasando de las tablas rígidas a modelos flexibles basados en documentos y colecciones. Asimismo, resolver problemas de sincronización mediante flujos de datos reactivos (Flows) ha sido clave para entender cómo funcionan las aplicaciones móviles modernas de alto rendimiento.

Entre las deudas técnicas identificadas para una versión de producción destacan: la implementación de migraciones explícitas en Room (actualmente usa *fallbackToDestructiveMigration* que elimina datos locales al actualizar), la sustitución del pago simulado por integración real con Stripe mediante un backend intermediario, y la ampliación de la cobertura de tests a las capas de repositorio y UI.

¹⁸ Base de datos que no usa tablas fijas, permitiendo guardar datos de forma más flexible y rápida, como documentos JSON.

9. Bibliografía

Acerca de los repositorios - GitHub Docs. (s. f.). GitHub Docs.

<https://docs.github.com/es/repositories/creating-and-managing-repositories/about-repositories>

Acerca del complemento de Android para Gradle. (s. f.). Android Developers.

<https://developer.android.com/build/releases/about-agp?hl=es-419>

Aplicación móvil : Stripe: ayuda y soporte. (s. f.). <https://support.stripe.com/topics/mobile-app?locale=es-ES>

Aspectos básicos. (s. f.). Firebase. <https://firebase.google.com/docs/guides?hl=es-419>

Cómo conectarse a Firebase. (s. f.). Android Developers.

<https://developer.android.com/studio/write/firebase?hl=es-419>

Cómo realizar pruebas. (s. f.). Android Developers.

<https://developer.android.com/jetpack/androidx/releases/test?hl=es-419>

Compose. (s. f.). Android Developers.

<https://developer.android.com/jetpack/androidx/releases/compose?hl=es-419>

Corrutinas de Kotlin en Android. (s. f.). Android Developers.

<https://developer.android.com/kotlin/coroutines?hl=es-419>

Descripción general de CameraX. (s. f.). Android Developers.

<https://developer.android.com/media/camera/camerax?hl=es-419>

Figma Learn - Centro de ayuda. (s. f.). <https://help.figma.com/hc/es-es>

Flujos de Kotlin en Android. (s. f.). Android Developers.

<https://developer.android.com/kotlin/flow?hl=es-419>

Guía de arquitectura de apps. (s. f.). Android Developers.

<https://developer.android.com/topic/architecture?hl=es-419>

Material Design 3 en Compose. (s. f.). Android Developers.

<https://developer.android.com/develop/ui/compose/designsystems/material3?hl=es-419>

Mijacobs. (s. f.). *¿Qué es Scrum? - Azure DevOps.* Microsoft Learn.

<https://learn.microsoft.com/es-es/devops/plan/what-is-scrum>

Navegación. (s. f.). Android Developers.

<https://developer.android.com/jetpack/androidx/releases/navigation?hl=es-419>

¿Qué es Scrum y cómo puede ayudar a tu empresa? | Cámara de España. (s. f.). Cámara de

España. <https://www.camara.es/blog/innovacion-y-competitividad/que-es-scrum-y-como-puede-ayudar-tu-empresa>

Recomendaciones para la arquitectura de Android. (s. f.). Android Developers.

<https://developer.android.com/topic/architecture/recommendations?hl=es-419>

Room. (s. f.). Android Developers.

<https://developer.android.com/jetpack/androidx/releases/room?hl=es-419>

Zxing. (s. f.). *GitHub - zxing/zxing: ZXing («Zebra Crossing») barcode scanning library for*

Java, Android. GitHub. <https://github.com/zxing/zxing>